CSE 4304-Data Structures Lab. Winter 2024-25
Date: 04 March 2026
Target Group: All groups
Topic: Range Queries: Fenwick Tree point update, Segment Tree.

Instructions:

- Task naming format: fullID_T01L01_2A.c/cpp
- If you find any issues in the problem description/test cases, comment in the Google Classroom.
- If you find any tricky test cases that I didn't include but that others might forget to handle, please comment! I'll be happy to add them.
- Use appropriate comments in your code. This will help you recall the solution more easily in the future.
- The obtained marks will vary based on the efficiency of the solution.
- Do not use <bits/stdc++.h> library.
- Modified sections will be marked with BLUE color.
- You are allowed to use the STL stack unless it's specifically mentioned to use manual functions.

Group	**Tasks**
1A/1B/2A/2B	1 2 3
Assignment	4

Task 1: Basic Min Segment Tree

Given an array with **N** elements (indexed from 1 to N) and a Query within range (i,j), your task is to find the minimum value of that range.

Input:

- Each test case will have two values (N, Q) in the first line, denoting the total number of elements N and the total number of queries Q.
- The following line will take N numbers (A segment tree will be built using these values).
- Each of the following Q lines will have two values indicating i & j for each range.

Output:
For each Query, you have to print the minimum value of the range (i,j).

Sample Input	**Sample Output**
5 3 78 1 22 12 3 1 2 3 5 4 4	 1 3 12
1 1 10 1 1	 10
6 6 20 50 10 40 90 30 1 6 3 3 5 5 5 6 4 6 3 6	 10 10 90 30 30 10

Note: The maximum complexity of each query should be in O(logN).

Task 2: Curious Robin Hood (solve using segment tree)

Robin Hood likes to loot rich people and help poor people with this money. Instead of mixing the money, he keeps n sacks to keep money from different sources separately. The sacks are numbered 1 to N.
With these sacks, he usually performs three types of tasks:

1. Give all the money of i^{th} sack to the poor, leaving it empty.
2. Add a new component (given in input) into the j^{th} sack.
3. Find the total amount of money from i^{th} sack to j^{th} sack.

Since he is not a programmer, he seeks your help.

Input:
Each test case will have two values N, Q in the first line, denoting the total number of elements (N) and the total number of queries(Q).

The following line will take N numbers.

Each of the following Q lines can have values as:

- 1 i (Give all money from ith sack to the poor and show the current status.)
- 2 i v (Add v amount of money to ith sack.)
- 3 i j (Find the total amount of money from ith to jth sack.)

Output:

- Type-1: Print the amount of money that the poor will receive.
- Type-2: Show the current status of the sacks after the update.
- Type-3: Print the total amount from i to j.

Sample Input	**Sample Output**
5 6 3 2 1 4 5 1 5 2 4 4 3 1 4 1 3 3 3 5 1 2	 5 (3 2 1 4 0) 3 2 1 8 0 14 1 (3 2 0 8 0) 8 2 (3 0 0 8 0)

Link: https://lightoj.com/problem/curious-robin-hood

Note: This task can be solved using different approaches. Today, lets do it using Segment Tree.

Task 3: Implement the point update operation of a Fenwick Tree

Given an array of N integers, find the sum of a range [L, R] using the Fenwick Tree (a.k.a Binary Indexed Trees) data structure.

You have to print the following information:

- Print the Tree after construction (used linear construction)
- Answer the result of each query.
- Show the status of the tree after the update operation

Input:

- The first line contains the value of N. The Second line provides the N integers. After getting this input, the segment tree will be constructed.
- The next line provides a number t, denoting the number of query/update tasks in hand.
- Following t lines will have instructions for the query/update operation.
    - Format for query: 1 begin end
    - Format for update: 2 index newValue (to be added to the existing value of the index)

Sample Input	**Sample output**
16 5 10 5 10 5 10 5 10 5 10 5 10 5 10 5 10 6 1 1 7 1 1 11 1 11 15 2 9 15 1 1 11 1 11 15	Status of Fenwick Tree (idx: value): 1:5 2:15 3:5 4:30 5:5 6:15 7:5 8:60 9:5 10:15 11:5 12:30 13:5 14:15 15:5 16:120 Query: Sum=50 Query: Sum=80 Query: Sum=35 Updated tree: 1:5 2:15 3:5 4:30 5:5 6:15 7:5 8:60 9:20 10:30 11:5 12:45 13:5 14:15 15:5 16:135 Query: Sum=95 Query: Sum=35

Note: We'll cover the point update in the assignment.

Task 4: Segment Tree lazy propagation

Implement the Query and Update operation for a **Min-Segment tree** maintaining the 'Lazy-propagation' property. Test the system for different cases and make sure everything is implemented correctly.
The first line contains N and Q, denoting the number of elements and the number of queries/updates. The following Q lines will contain query/update operations.

- Update is denoted by 2. Format: 2 i j x. You have to add x with the value that is present at the node.
- Query is denoted by 1. Format: 1 i j. You have to return the minimum value within the range (i, j)

Print the status of the segment tree and lazy tree after each operation.

Sample Input	**Sample Output**
8 4 -1 2 4 1 7 1 3 2 2 1 4 3 2 1 4 1 2 2 2 2 1 4 6	-1 -1 1 -1 1 1 2 -1 2 4 1 7 1 3 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 **1 2** 1 -1 1 1 2 -1 2 4 1 7 1 3 2 0 0 0 **3 3** 0 0 0 0 0 0 0 0 0 0 1 **3** 1 -1 1 1 2 -1 2 4 1 7 1 3 2 0 0 0 **4 4** 0 0 0 0 0 0 0 0 0 0 1 **3** 1 **3 5** 1 2 **3 8** 4 1 7 1 3 2 0 0 0 **0 0** 0 0 0 0 **4 4** 0 0 0 0 1 1 3 1 3 5 1 2 3 8 **8 5** 7 1 3 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0